3. Final Report

1021 EECS by: Professor Mohaghegh || April 24th, 2025 || Maximus A., Ian H., Eric X., Kevin Y.

3.1 Introduction

Problem Definition

As students who often get **bored** due to tedious tasks and downtime between lectures in University, we decided to create the game of **Snake** for our summative telemetry project to solve this boredom. In the creation of the code, we had the restriction of only using two push buttons, making us improvise a solution to the normalized four buttons used in the game of Snake. As a result, we used our 1025 Linear Algebra knowledge to create a rotation matrix for the snake to move. This game requires precise timing, constant input polling, and real-time display updates, making it an ideal showcase for practical embedded engineering.

Project Objectives

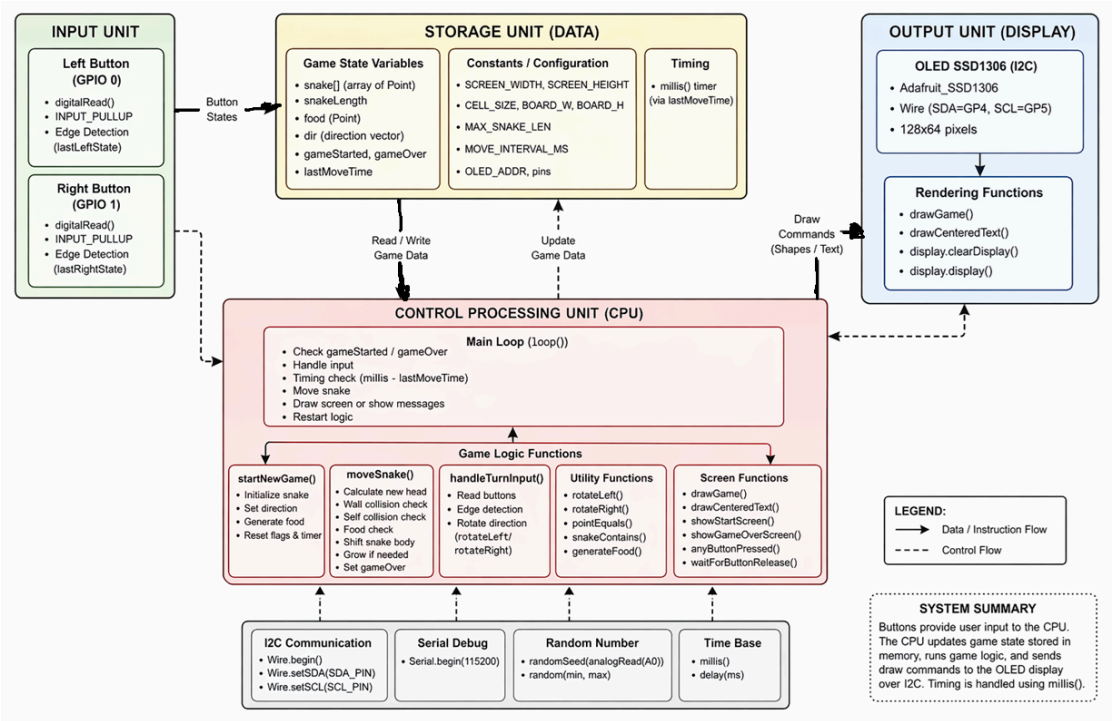
Our primary goal was to build a responsive, smooth-running **Snake** game on a microcontroller like the Raspberry Pi Pico 2W. We wanted to demonstrate how our firmware can juggle user inputs and graphical rendering simultaneously without lag. This project highlights the system's ability to manage dynamic state changes, like the snake growing in size, while adhering to strict timing constraints.

System Overview

The setup features a Raspberry Pi Pico 2W paired with an OLED display. Players navigate the snake using two directional buttons; one button is used to turn leftwards, and the other is used to turn rightwards. As the snake moves across the grid, it consumes food to grow,. while the system constantly monitors for "game over" conditions, such as the head of the snake hitting a wall (going out of bounds), hitting the snake’s own body, or the snake completely filling the play space.

3.2 System Architecture

High-Level block diagram



Data Flow Description

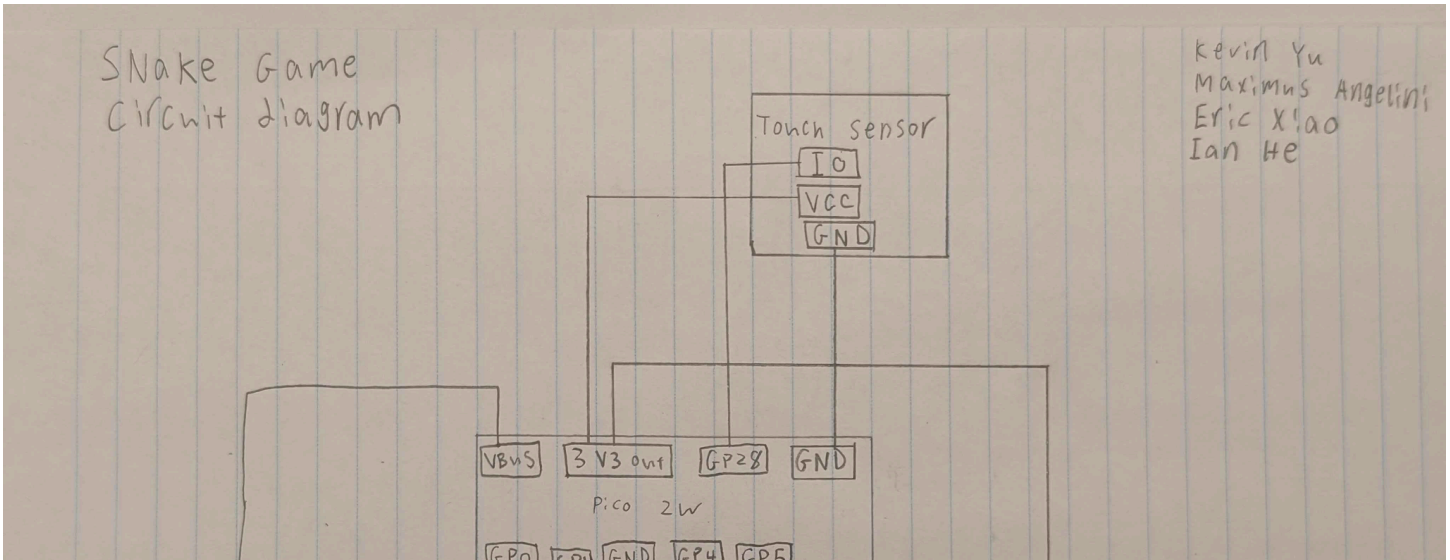
When a player hits a button, a digital signal travels to the Pico 2W. The firmware interprets this pulse and updates the snake's heading. At set intervals, the game engine calculates the snake's new coordinates and checks for collisions within the main loop. Finally, the updated "map" is sent to the OLED display. This loop runs rapidly to ensure the movement looks fluid to the human eye.

3.3 Hardware Design

Components Used

We kept the hardware streamlined: a **Raspberry Pi Pico**, an **SSD1306 OLED display**, and two tactile buttons. We used the **I2C protocol** for the display, which significantly reduced the amount of wiring needed.

Circuit Design



Design Decisions and Trade-offs

We chose the OLED for its high contrast and low power draw. While using buttons was a practical choice for our current kit, we recognize that a joystick would offer more fluid control. Additionally, the small resolution of the screen (128px by 64px) means we had to keep our graphics simple and grid-based.

3.4 Software Design

Embedded Firmware Design

The firmware is modular, separating the code into three distinct tasks: input, logic, and rendering. We implemented **edge detection** for the inputs to ensure that a single button press doesn't register as multiple turns.

Code Structure

We used a Point structure to manage the x and y coordinates about the OLED screen, written in the form (x, y). The snake itself is stored as an array of these points, allowing it to "grow" as it eats. By organizing the code into modular functions like moveSnake() and drawGame(), we kept the logic clean and easy to troubleshoot.

Libraries Used

We relied on the **Adafruit GFX** and **Adafruit SSD1306** libraries. These handled the heavy lifting of pixel-by-pixel communication, allowing us to focus on the actual game mechanics rather than low-level driver code. We also used the Wire.h library which is the Arduino I2C communication library, which contains I2C that is the communication method used by your OLED display.

3.5 Implementation Details

Integration Challenges and Solutions

- **The "Spinning" Problem:** Initially, holding a button down caused the snake to turn repeatedly and instantly cause the snake head to hit its own body. We solved this with **edge detection**, by making the game now only reacts to the moment the button is first pressed.
- **Food Placement:** We had to ensure food didn't spawn "inside" the snake's body. We fixed this by adding a validation check that scans the snake’s current coordinates before placing a new food item.

3.6 System Behavior and Operation

Timing and Synchronization

To keep the game responsive, we avoided using "blocking" delays. Instead, we used the millis() function to track elapsed time. This allows the snake to move at a consistent speed while the processor remains free to check for button presses every few microseconds, especially because snake is a game which needs fast reaction time and adaptiveness depending on where the apple is located.

Data acquisition and processing pipeline

User inputs are recorded from two digital pin inputs corresponding to the two buttons used to turn the snakes' head. These pins send signals to the Raspberry Pi Pico 2W board which will increment the snake in response to an input, or lack thereof. This logic is then pushed to the OLED screen to be drawn, causing the snake head to turn or continue moving in a straight-line depending on if the user input was detected. In the case of the game state changing,the OLED screen will instead draw the game over screen. This process is then repeated again until the user unplugs the system.

3.7 Results and Evaluation

The result is a stable, fully playable game. During testing, we focused on "stress tests," such as rapid-fire button presses and navigating the snake into tight corners. The system proved to be reliable, with zero noticeable input lag and consistent frame rates. Additionally, we tested each end-condition, such as if hitting the "walls" of the screen or the snakes' body would end the game prematurely with the game over screen.

3.8 Strengths and Weaknesses

- **Strengths:** The modular code makes it very easy to add new features or port the game to different hardware. To add on, the firmware design is very minimalistic thanks to the rotation matrix inspiration reducing the required sensors from 4 buttons to move in 4 directions, to 2 buttons to move in 4 directions. Finally, the software implementation uses delta time, ensuring that the system updates at consistent intervals regardless of the processing power of the hardware.
- **Weaknesses:** The current version lacks "extra" polish, like a scoring system or sound effects. The small screen also limits how long a game can last before the snake fills up the entire grid. Additionally, many of the collision check methods use busy-loops to iterate through the snake array. While the small screen size limits how large this array gets, at a certain point this check will significantly slow down the system. This could be alleviated with use of a separate thread to check for collisions instead of using busy loops.

3.9 Assumptions

The system assumes stable power from the USB connection and reliable button inputs. It also assumes that the I²C communication speed is sufficient for smooth display updates, which was confirmed during testing.

3.10 Future Improvements

If we were to expand this project, our first steps would be:

1. Adding audio feedback to improve game feel and user experience.
2. Implementing a **high-score system** and leaderboard using Pico's internal memory.
3. Pushing the leaderboard to an external database in order to save leaderboard memory
4. Implementing threads to iterate through snake body for collision checks instead of busy loops within the main thread to reduce processing time at larger snake sizes.

3.11 Individual Contributions

This was a collaborative effort. Everybody engaged in the project to a significant degree. Everyone contributed to the coding, Maximus and Eric X especially. Kevin and Maximus build the physical circuit/design for our program to function. Additionally, Kevin bought the components needed for our snake game to work (i.e. the OLED screen, buttons, etc). We all contributed this lab report, especially Ian He who helped compile the work into an orderly fashion as well for the video editing portion. We all presented our own sections of the program which we researched the most, when creating it. Maximus presented the first quarter, while Ian did the next, then Eric X, and lastly Kevin to end off the video.